CAMERA-BASED HANDWRITING RECOGNITION

Implementation Specification for Antigravity

Classical Computer Vision + Shallow MLP
No deep learning models

Purpose. Use this PDF as the project brief when asking Antigravity to implement the system. Build each stage as a separate, testable module and document algorithmic assumptions.

Input: a static camera capture or phone photograph (JPEG/PNG). **Output:** reconstructed text in a UTF-8 .txt file and an annotated copy of the original image.

Core constraint

Use traditional image processing and a shallow, fully connected Multi-Layer Perceptron (MLP) only. Do not use CNNs, RNNs, LSTMs, Vision Transformers, pretrained vision-language models, OCR APIs, or any other deep-learning-based recognition system.

1. Objective and pipeline overview

Create an end-to-end handwriting recognition prototype that corrects image geometry and lighting, segments writing hierarchically, extracts handcrafted features, classifies character crops with an MLP, and exports predicted text with visual annotations.

INPUT

Static camera / phone image
JPEG or PNG

PHASE 1

Image ingestion
Optional 4-point perspective transform

PHASE 2

Grayscale, background division,
CLAHE, adaptive binarization

PHASE 3

Line → word → character segmentation
Noise and scratch rejection

PHASE 4

Slant estimation, affine shear,
normalization to 16 × 16

PHASE 5

Circular DDD handcrafted
feature extraction

PHASE 6

Feature scaling → shallow MLP
→ class scores / softmax

PHASE 7

Text reconstruction + .txt export
Annotated image export

Keep intermediate images and bounding-box overlays available for debugging. Preserve the original image for final annotations.

2. Phase-by-phase requirements

Phase 1 — Image ingestion and document flattening

- Load images with `cv2.imread`; report unreadable files clearly.
- Make the four-point perspective transform optional. If page corners cannot be detected reliably, allow the user to skip it or supply corners manually.
- Keep an untouched copy of the original photo for annotations.

Phase 2 — Illumination-robust preprocessing

- Convert to grayscale.
- Estimate uneven illumination using a morphological background estimate and safe division/normalization; prevent division by zero.
- Apply CLAHE for local contrast enhancement.
- Binarize with Sauvola if implemented, or Gaussian adaptive thresholding as a fallback. Expose key parameters in configuration.

Phase 3 — Hierarchical segmentation

- Detect lines using horizontal projection profiles.
- Group words using vertical projection gaps and/or horizontal dilation; preserve top-to-bottom and left-to-right order.
- Extract character candidate boxes and reject tiny components using pixel mass/area thresholds.
- Save debug overlays for line, word, and character boxes. Projection segmentation may struggle with joined cursive writing, so expose tunable thresholds.

Phase 4 — Geometric normalization

- Estimate slant using the proposed dual-centroid method. Document the exact centroid definitions and angle formula.
- Apply affine horizontal shear to reduce slant without cropping strokes.
- Normalize every character crop to a consistent 16 × 16 grayscale grid using consistent padding and resampling.

Phase 5 — Circular DDD handcrafted feature engine

- Implement virtual circular-map tiling / wrap-around and 8-direction dual-set ray casting in a separate, testable module.
- Use W-set for stroke-related features and B-set for thickness-related features, according to the project's intended design.
- Document ray-casting rules, boundary behavior, feature order, normalization, and output dimension. Do not invent unspecified formulas; leave a clear TODO or request clarification where needed.

Phase 6 — Shallow MLP classifier

- Use a consistent feature scaler fitted on training data or a documented [0, 1] transformation.
- Use a fully connected MLP architecture of 256 → 100 → 90 → N classes, with N and label mapping determined by the chosen EMNIST split.
- Use a conventional shallow implementation such as scikit-learn MLPClassifier or a small NumPy implementation. Do not use deep-learning frameworks or convolutional layers.
- Train and evaluate on a documented EMNIST alphanumeric split. Save the model, scaler, and label map. Treat output confidence as a score, not a guarantee of correctness.

Phase 7 — Reconstruction and export

- Reconstruct characters into words and lines in reading order.
- Export the recognized text as UTF-8 .txt.
- Create an annotated image from the original photo with bounding boxes, predicted labels, and optional confidence scores.
- Show a run summary with counts of detected lines, words, and characters, output paths, and warnings.

3. Design issues to resolve before implementation

Feature-dimension mismatch

A 16 × 16 image contains 256 pixel values. But averaging it into a 4 × 4 grid of spatial blocks produces 16 values (one average per block), not a 256-dimensional vector. The supplied design mentions both “4×4 spatial block averaging” and a “256-dimensional invariant vector”; these statements conflict.

Recommended decision: if the MLP input must remain 256, flatten the 16 × 16 normalized image or define a documented 256-feature DDD descriptor. If the 4 × 4 block averages are intended, update the MLP input size to 16 or combine them with additional features. Do not silently mix these options.

- **DDD definition:** the supplied pipeline does not fully define the Circular DDD equations/ray rules. Implement only after those rules are specified, or clearly label the module as a placeholder.
- **EMNIST labels:** verify the selected split's classes and exact label-index mapping before decoding predictions.
- **Segmentation limitations:** projection profiles work best when lines and characters are separated. Save debug images and report failures instead of silently producing misleading text.
- **Perspective transform:** do not force a four-corner transform when no page boundary is visible.
- **Evaluation:** report character-level accuracy and, where possible, test on handwritten images not used for training.

4. Suggested project structure

```
handwriting-recognition/
■■■ main.py              # application entry point
■■■ config.py            # thresholds and paths
■■■ ingestion.py         # image load and perspective warp
■■■ preprocessing.py     # illumination, CLAHE, binarization
■■■ segmentation.py      # lines, words, character boxes
■■■ normalization.py     # slant correction and 16×16 resize
■■■ features.py          # Circular DDD feature extraction
■■■ train_mlp.py         # MLP training and evaluation
■■■ predict.py           # inference and label decoding
■■■ reconstruction.py    # text ordering and .txt export
■■■ visualization.py     # debug views and annotated output
■■■ models/              # saved model, scaler, label map
■■■ outputs/             # text, annotated, and debug images
■■■ tests/               # stage-level unit tests
■■■ requirements.txt
■■■ README.md
```

Antigravity may adjust filenames if module responsibilities remain clear.

5. Acceptance criteria

Area	Acceptance criterion
Image loading	Valid JPEG/PNG loads; invalid or corrupt files produce a readable error.
Preprocessing	Intermediate grayscale, background-corrected, contrast-enhanced, and binary images can be inspected.
Segmentation	Bounding boxes are visualized and saved; output order is deterministic.
Normalization	Every accepted character crop becomes a consistent 16 × 16 representation.
Features	Feature output is finite, deterministic, and exactly the documented dimension.
MLP	Training and inference share the same feature pipeline, scaler, and label map.
Exports	UTF-8 text and annotated original image are generated.
No deep learning	No CNN/RNN/Transformer or deep-learning OCR model appears in code or dependencies.

Area	Acceptance criterion
Reproducibility	README includes setup, training, inference commands, dataset instructions, and limitations.

6. Instructions for Antigravity

- Inspect the existing project folder before creating or overwriting files.
- Implement incrementally: image loading, preprocessing, segmentation, normalization, feature extraction, MLP training/inference, then export.
- Run a small test after each stage and save intermediate debug outputs.
- Ask for clarification or leave a clearly marked TODO when the DDD algorithm or feature dimension is underspecified.
- Provide exact terminal commands for dependency installation, training, and running inference on an image.
- Keep the code student-friendly, modular, and documented. Include setup and known limitations in README.md.

Reference note

The project owner listed Wong & Sebastian (2015), Katiyar & Mehfuz (2015), and Oh & Suen (1998) as foundational references. Verify full bibliographic details before citing them in the final report, and explain which design choice each source supports.

End of implementation brief